

Roll no :- S002	Name :- Kaif Ansari
Batch :- 1	Class :- SY.BSC.IT
Date of assignment :- 22/09/2026	Date of submission:-22/09/2026

PRACTICAL 27

Aim:- Implement one-hot encoding for a categorical column in a dataset.

Input:-

```

1  object Practical27 extends App
2  {
3      val data = List("Red", "Blue", "Green", "Red", "Blue")
4      for (value <- data)
5      {
6          val r = if (value == "Red") 1 else 0
7          val b = if (value == "Blue") 1 else 0
8          val g = if (value == "Green") 1 else 0
9          println(value + " -> Red:" + r + " Blue:" + b + " Green:" + g)
10     }
11 }
12

```

Output:-

```

Run  Practical27 x
C:\Users\syed usaid\.jdk\ms-17.0.20.1
Red -> Red:1 Blue:0 Green:0
Blue -> Red:0 Blue:1 Green:0
Green -> Red:0 Blue:0 Green:1
Red -> Red:1 Blue:0 Green:0
Blue -> Red:0 Blue:1 Green:0
Process finished with exit code 0
|

```

PRACTICAL 28

Aim:- Compute the euclidean distance between two Breeze vector. Use it for nearest neighbour classification.

Input:-

```
build.sbt x Practical27.scala
1 ThisBuild / scalaVersion := "2.12.18"
2
3 libraryDependencies += "org.scalanlp" %% "breeze" % "2.1.0"
4
5 lazy val root = (project in file("."))
6   .settings(
7     name := "Practical 27"
8   )
9
```

```
build.sbt Practical27.scala x
2
3 object Demo extends App
4 {
5   val A = DenseVector(2.0, 3.0)
6   val B = DenseVector(5.0, 7.0)
7
8   val distance = norm(A - B)
9
10  println("Euclidean Distance: " + distance)
11
12  val C = DenseVector(2.5, 3.5)
13
14  val d1 = norm(C - A)
15  val d2 = norm(C - B)
16
17  if (d1 < d2)
18    println("Nearest Neighbor: A")
19  else
20    println("Nearest Neighbor: B")
21 }
```

Output:-

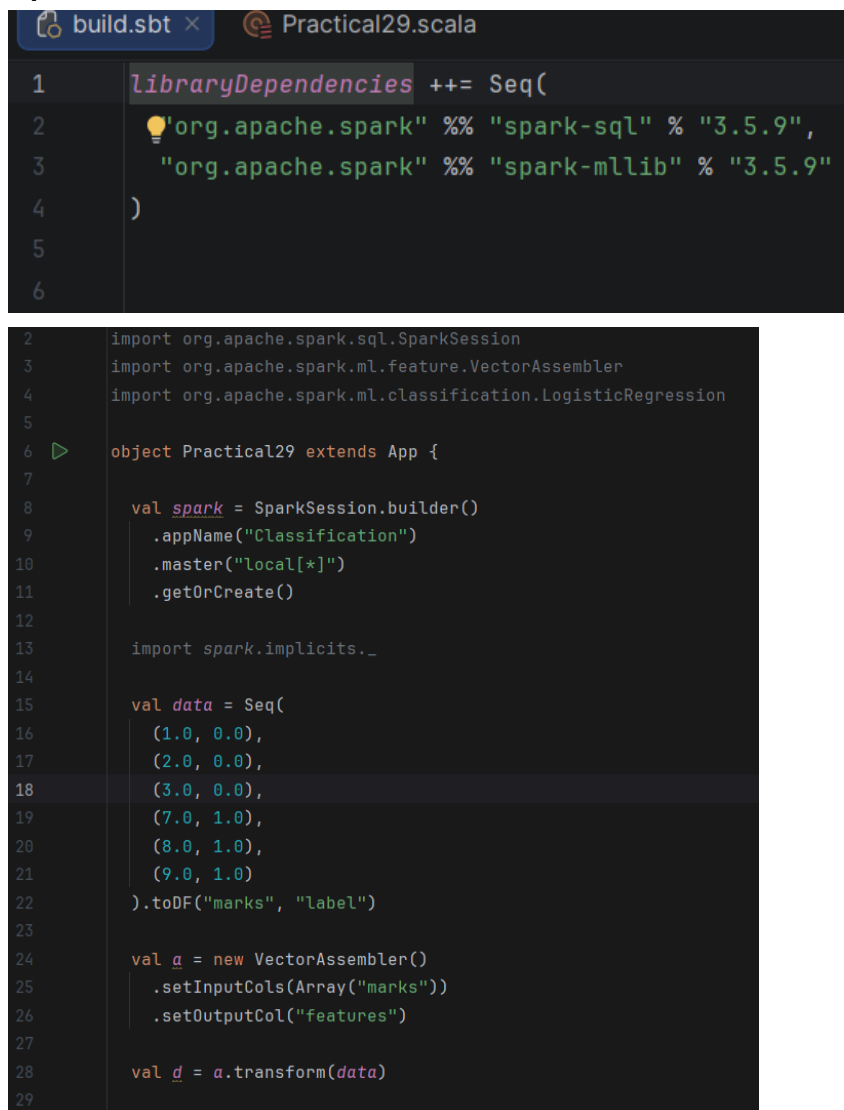
```
Euclidean Distance: 5.0
Nearest Neighbor: A

Process finished with exit code 0
```

PRACTICAL 29

Aim:- Create a simple spark MLlib pipeline to classify data. Use logistic regression or decision trees.

Input:-



```
build.sbt x Practical29.scala

1 libraryDependencies += Seq(
2     'org.apache.spark' %% "spark-sql" % "3.5.9",
3     "org.apache.spark" %% "spark-mllib" % "3.5.9"
4 )
5
6

2 import org.apache.spark.sql.SparkSession
3 import org.apache.spark.ml.feature.VectorAssembler
4 import org.apache.spark.ml.classification.LogisticRegression
5
6 object Practical29 extends App {
7
8     val spark = SparkSession.builder()
9         .appName("Classification")
10        .master("local[*]")
11        .getOrCreate()
12
13    import spark.implicits._
14
15    val data = Seq(
16        (1.0, 0.0),
17        (2.0, 0.0),
18        (3.0, 0.0),
19        (7.0, 1.0),
20        (8.0, 1.0),
21        (9.0, 1.0)
22    ).toDF("marks", "label")
23
24    val a = new VectorAssembler()
25        .setInputCols(Array("marks"))
26        .setOutputCol("features")
27
28    val d = a.transform(data)
29
```

```

29
30     val model = new LogisticRegression()
31     .setLabelCol("label")
32     .setFeaturesCol("features")
33
34     val result = model.fit(d)
35
36     val prediction = result.transform(d)
37
38     prediction.select("marks", "label", "prediction").show()
39
40     spark.stop()
41 }
42

```

Output:-

```

+-----+-----+-----+
|marks|label|prediction|
+-----+-----+-----+
|  1.0|  0.0|      0.0|
|  2.0|  0.0|      0.0|
|  3.0|  0.0|      0.0|
|  7.0|  1.0|      1.0|
|  8.0|  1.0|      1.0|
|  9.0|  1.0|      1.0|
+-----+-----+-----+

```